

Risk Warnings Extraction Engine

Automated Detection of Financial Risk Signals in Annual Reports

JP Morgan — Internship Project Submission

April 6, 2026

Abstract

This report presents a systematic methodology for extracting risk warnings from corporate annual reports using a multi-stage large language model (LLM) pipeline. We identify the most informative sections of an annual report for credit risk assessment, design a structured extraction engine, and validate the approach against a curated set of annual reports from companies that subsequently filed for bankruptcy. The pipeline is designed for scalability and is well-suited to the volume of documents processed in a banking context.

Contents

1	Question I: Extracting Important Risk Warning Points from Annual Reports	3
1.1	Overview	3
1.2	Taxonomy of Auditor Opinion Types	3
1.3	Emphasis of Matter and Going Concern	3
2	Question II: Other Important Sections in an Annual Report	4
2.1	Prioritised Section Map	4
2.2	Notes to Financial Statements	5
2.3	Cash Flow Statement	5
2.4	Management Discussion & Analysis	5
2.5	Risk Factors	6
2.6	Auditor or Director Changes	6
3	Question III: Automatic Extraction Engine	6
3.1	Pipeline Architecture	6
3.1.1	Stage 0 — PDF Parsing and Section Prioritisation	7
3.1.2	Stage 1 — Coarse Page Flagging	7
3.1.3	Stage 2 — Category-Specific Extraction	8
3.1.4	Stage 3 — Synthesis and Risk Memo Generation	11
3.2	Full Orchestration	11
3.3	Design Decisions and Limitations	12
4	Question IV: Validation on Bankrupt Companies	12
4.1	Test Dataset	12
4.2	Results by Company	13
4.2.1	Case 1: Evergrande Group (2022 Annual Report)	13
4.2.2	Case 2: Silicon Valley Bank (2022 Annual Report)	15
4.2.3	Case 3: Bed Bath & Beyond (2022 Annual Report)	16
4.2.4	Case 4: Lehman Brothers (2007 Annual Report)	16
4.2.5	Case 5: Wirecard AG (2018 Annual Report)	17
4.3	Aggregate Results Summary	18
4.4	Discussion	20
5	Conclusion	21
A	Appendix: Full Codebase	22
A.1	risk_categories.py	22
A.2	models.py — Pydantic validation schemas	22
A.3	page_flagger.py — Stage 1	23
A.4	category_extraction.py — Stage 2	25
A.5	synthesiser.py — Stage 3	25
A.6	risk_extractor.py — Pipeline orchestrator	26
A.7	usage_tracker.py	27
B	Appendix: Sample LLM Outputs	28
B.1	Stage 1 Output: Flagged Pages	28

B.2	Stage 2 Output: Contingent Liabilities Extraction	28
B.3	Stage 2 Output: Cash Flow Warnings Extraction	28
B.4	Pipeline Usage Summary	28
C	Appendix: Annual Report Sources	29

Question I: Extracting Important Risk Warning Points from Annual Reports

Overview

Annual reports are lengthy documents, often spanning 150–300 pages, the majority of which contain routine disclosures, boilerplate governance statements, and marketing-oriented narrative. The challenge in a credit risk context is to efficiently identify the minority of content that carries genuine signal about a company’s financial health and future viability.

One of the most critical sections is the **Auditor’s Report**, which contains the independent auditor’s formal opinion on the financial statements. Deviations from a clean, unqualified opinion are among the strongest quantifiable warning signals available to analysts. The Hong Kong Institute of Certified Public Accountants (HKICPA) provides reference examples of auditor opinion language, including qualified opinions, at:

<https://www.hkicpa.org.hk/.../SME-FRS-Jul2009.pdf>

Taxonomy of Auditor Opinion Types

The four principal types of auditor opinion, ranked by severity, are summarised below:

Opinion Type	Meaning
Unqualified	Clean opinion; financials present a true and fair view
Qualified	Auditor has reservations on a specific, material but not pervasive issue
Adverse	Financials are materially misstated in a pervasive way
Disclaimer	Auditor was unable to obtain sufficient evidence to form an opinion

Table 1: Auditor opinion types ranked by severity

Any opinion type other than *Unqualified* should be treated as an immediate red flag in credit assessment.

Emphasis of Matter and Going Concern

Even within an unqualified opinion, auditors may include an **Emphasis of Matter** paragraph that draws attention to a significant uncertainty without modifying the overall opinion. The most consequential is a **going concern** qualification, which signals that the auditor has material doubt about the company’s ability to continue operating for the next twelve months.

Key phrases to detect include:

- “material uncertainty related to going concern”

- “significant doubt about the entity’s ability to continue”
- “except for the effects of” (qualifier language)
- “we were unable to obtain sufficient appropriate audit evidence”

Clean Unqualified Opinion

“In our opinion, the consolidated financial statements give a true and fair view of the financial position of the Group as at 31 December 20XX, and of its financial performance and cash flows for the year then ended in accordance with Hong Kong Financial Reporting Standards and have been properly prepared in compliance with the Hong Kong Companies Ordinance.”

Disclaimer of Opinion with Going Concern

“We do not express an opinion on the consolidated financial statements of the Group. Because of the significance of the matters described in the Basis for Disclaimer of Opinion section of our report, we have not been able to obtain sufficient appropriate audit evidence to provide a basis for an audit opinion. [...] These conditions indicate the existence of material uncertainties which may cast significant doubt about the Group’s ability to continue as a going concern.”

Left: standard language from HKICPA reference templates. Right: representative language from annual reports containing going concern qualifications (adapted from publicly filed Hong Kong listed company reports).

Question II: Other Important Sections in an Annual Report

Prioritised Section Map

Beyond the Auditor’s Report, the following sections contain the highest density of risk-relevant content. They are ranked by the degree of independent verification (i.e., the further from management-authored content, the more reliable the signal):

Rank	Section	Key Risk Signals
1	Auditor's Report	Opinion type, going concern, emphasis of matter
2	Notes to Financial Statements	Contingent liabilities, covenant conditions, related-party transactions, accounting policy changes
3	Cash Flow Statement	Divergence between net income and operating cash flow; sustained negative operating cash flow
4	MD&A	Vague or defensive language; repeated external blame; new risks; contradictions with financial figures
5	Risk Factors	New or upgraded risks versus prior year
6	Directors' Report	Mid-year resignations; large insider share sales

Table 2: Annual report sections ranked by risk signal reliability

Notes to Financial Statements

The notes are the richest secondary source of risk data after the auditor's report. Specific items to extract include:

- **Contingent liabilities:** Pending litigation, regulatory investigations, or tax disputes that may crystallise into real losses
- **Debt covenants:** Conditions attached to borrowings; proximity to breach is a major liquidity warning
- **Related-party transactions:** Money flows between the company and its directors or controlling shareholders, which may indicate tunnelling or conflicts of interest
- **Accounting policy changes:** Shifts in recognition, measurement, or presentation that can artificially improve reported performance

Cash Flow Statement

The cash flow statement is difficult to manipulate relative to the income statement, making it a key cross-check. A company reporting strong net income while generating negative or declining operating cash flow over multiple periods is a classic **earnings quality** warning.

Management Discussion & Analysis

The MD&A is authored by management and should be read critically. Linguistic signals of concern include:

- Repeated attribution of underperformance to external factors (“macro headwinds”, “FX impact”, “one-off items”)

- Vague language around declining metrics without quantification
- Introduction of new non-GAAP measures that exclude deteriorating items
- Forward guidance that is inconsistent with the balance sheet position

Risk Factors

The Risk Factors section is management-authored and lists the principal risks the company faces. In isolation, a single year’s disclosure is often boilerplate. The real signal emerges from **year-on-year comparison**: new risks that did not appear in the prior filing, risks whose severity language has escalated, and previously disclosed risks that have been quietly removed.

	FY 2021 Disclosure	FY 2022 Disclosure
Liquidity Risk	“The Group manages its liquidity position to ensure sufficient cash flows are available to meet obligations as they fall due.”	“The Group has been unable to repay certain borrowings as they fell due. As at 31 December 2022, overdue borrowings amounted to RMB 389.8 billion.”
Severity	Routine disclosure (low signal)	Explicit admission of default (critical signal)

Table 3: Year-on-year risk factor escalation: the same disclosure category can shift from routine to critical between consecutive annual reports.

The extraction engine (described in Section III) can detect such escalations by running category-specific extraction on consecutive annual reports for the same company and comparing the structured output. For example, a `debt_covenants` result that returns `compliance_status: ``in compliance``` in year $t-1$ but `compliance_status: ``in breach``` in year t represents a programmatically detectable escalation.

Auditor or Director Changes

A change of auditor, particularly to a lesser-known firm or one occurring outside the normal rotation cycle, may indicate the previous auditor declined to continue. Similarly, the sudden resignation of a CFO or multiple directors is a strong insider-knowledge signal.

Question III: Automatic Extraction Engine

Pipeline Architecture

The extraction engine follows a four-stage pipeline modelled on a **coarse filter** → **targeted extraction** → **synthesis** design. This architecture manages context window constraints while ensuring that expensive LLM extraction calls are applied only to high-signal pages.

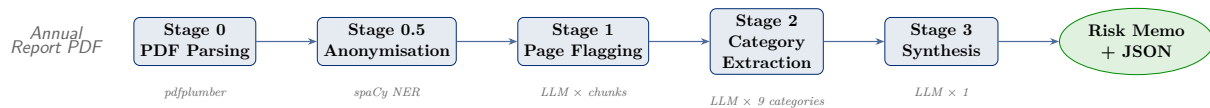


Figure 1: Four-stage extraction pipeline. LLM calls occur at Stages 1, 2, and 3. Stage 2 runs in parallel across detected categories using a thread pool.

3.1.1 Stage 0 — PDF Parsing and Section Prioritisation

The annual report PDF is parsed into a dictionary of page-indexed text blocks using PyMuPDF. Where an embedded table of contents (TOC) is available, sections are identified and mapped to page ranges. High-priority sections (auditor's report, notes, MD&A, risk factors) are flagged for priority processing.

```

1 import fitz # PyMuPDF
2
3 def extract_pages(pdf_path: str) -> dict[int, str]:
4     doc = fitz.open(pdf_path)
5     return {i + 1: page.get_text() for i, page in enumerate(doc)}
6
7 def extract_toc(pdf_path: str) -> list[dict]:
8     doc = fitz.open(pdf_path)
9     toc = doc.get_toc() # [[level, title, page], ...]
10    return [{"level": t[0], "title": t[1], "page": t[2]} for t in toc]
11
12 PRIORITY_KEYWORDS = [
13     "auditor", "independent", "going concern",
14     "risk", "director", "management discussion",
15     "notes to", "contingent", "litigation"
16 ]
17
18 def prioritise_sections(toc: list[dict]) -> list[dict]:
19     return [
20         item for item in toc
21         if any(kw in item["title"].lower() for kw in PRIORITY_KEYWORDS)
22     ]

```

Listing 1: Stage 0: PDF parsing and TOC extraction

Pages are then grouped into chunks of approximately 30 pages, with [PAGE N] markers embedded in the text to allow the LLM to reference exact page numbers.

3.1.2 Stage 1 — Coarse Page Flagging

Each chunk is submitted to the LLM with a structured prompt instructing it to identify pages containing content related to any of the nine risk warning categories. The output is a JSON array of flagged pages with associated categories.

```

1 CATEGORIES = [
2     "auditor_opinion",
3     "going_concern",
4     "contingent_liabilities",
5     "debt_covenants",
6     "related_party",

```

```

7     "accounting_policy",
8     "director_changes",
9     "cash_flow_warnings",
10    "md_and_a_red_flags"
11 ]
12
13 SYSTEM_PROMPT = """You are a financial risk analyst reviewing an annual
    report.
14 Identify pages containing potential risk warnings. Respond in valid JSON
    only."""
15
16 USER_TEMPLATE = """
17 Review pages {start} to {end} of an annual report.
18 For each page containing content related to any risk warning category,
    return:
19 [
20     [{"page": 12, "category": "going_concern", "reason": "brief explanation
        "}]
21 ]
22 Categories: {categories}
23 If no relevant pages found, return: []
24 PAGES:
25 {text}
26 """

```

Listing 2: Stage 1: Coarse page flagging prompt

3.1.3 Stage 2 — Category-Specific Extraction

Flagged pages are grouped by category. Each group is submitted with a category-specific extraction prompt that requests structured JSON output aligned to a predefined schema. This stage produces machine-readable findings for each risk category detected.

The full extraction prompts used in the pipeline are listed below. Each prompt instructs the LLM to return structured JSON aligned to a category-specific schema. All prompts share a common preamble (“You are an expert financial analyst...Respond ONLY with valid JSON”) and close with the flagged page content.

```

1 Extract:
2 - Type of opinion issued (unqualified/qualified/adverse/disclaimer)
3 - Exact basis for qualification if any
4 - Any emphasis of matter paragraphs
5 - Any going concern language
6
7 Return as JSON:
8 {"opinion_type": "...", "qualification_basis": "...",
9  "emphasis_of_matter": "...", "going_concern_language": "..."}
10 Use null for fields not found in the text.

```

Listing 3: Extraction prompt: Auditor Opinion

```

1 Extract all going concern disclosures:
2 - Exact language used
3 - Time period referenced
4 - Mitigating actions mentioned by management
5
6 Return as JSON:

```

```

7 {"disclosure_text": "...", "time_period": "...",
8   "mitigating_actions": "..."}
9 Use null for fields not found in the text.

```

Listing 4: Extraction prompt: Going Concern

```

1 Extract all contingent liabilities:
2 - Nature (lawsuit, regulatory, tax dispute etc)
3 - Estimated financial exposure if stated
4 - Likelihood assessment if stated
5 - Expected resolution timeline if stated
6
7 Return as JSON array:
8 [{"nature": "...", "exposure": "...", "likelihood": "...",
9   "timeline": "..."}]
10 Use null for fields not found. Return [] if none found.

```

Listing 5: Extraction prompt: Contingent Liabilities

```

1 Extract all debt covenant information:
2 - Covenant conditions described
3 - Current compliance status
4 - Any breaches or waivers mentioned
5 - Financial headroom if quantified
6
7 Return as JSON array:
8 [{"covenant_type": "...", "condition": "...",
9   "compliance_status": "...", "headroom": "..."}]
10 Use null for fields not found. Return [] if none found.

```

Listing 6: Extraction prompt: Debt Covenants

```

1 Extract all related-party transactions:
2 - Counterparty and their relationship to the company
3 - Nature and value of transaction
4 - Whether approved by independent directors
5
6 Return as JSON array:
7 [{"counterparty": "...", "relationship": "...",
8   "transaction_nature": "...", "value": "...",
9   "independent_approval": "..."}]
10 Use null for fields not found. Return [] if none found.

```

Listing 7: Extraction prompt: Related Party Transactions

```

1 Extract any changes in accounting policies:
2 - Policy that changed
3 - Reason given for change
4 - Financial impact of the change
5 - Whether comparatives were restated
6
7 Return as JSON array:
8 [{"policy_changed": "...", "reason": "...",
9   "financial_impact": "...", "restatement": "..."}]
10 Use null for fields not found. Return [] if none found.

```

Listing 8: Extraction prompt: Accounting Policy Changes

```

1 Extract any director or senior management changes:
2 - Name and role
3 - Nature of change (resignation, appointment, share sale)
4 - Timing
5 - Reason given if any
6
7 Return as JSON array:
8 [{"name": "...", "role": "...", "change_type": "...",
9   "timing": "...", "reason": "..."}]
10 Use null for fields not found. Return [] if none found.

```

Listing 9: Extraction prompt: Director Changes

```

1 Extract cash flow warning signals:
2 - Operating cash flow vs net income (are they diverging?)
3 - Any negative operating cash flow
4 - Large investing outflows
5 - Financing activities suggesting cash stress
6   (new debt, equity raises)
7
8 Return as JSON:
9 {"operating_cf": "...", "net_income": "...",
10  "divergence_flag": true/false, "key_observations": "..."}
11 Use null for fields not found.

```

Listing 10: Extraction prompt: Cash Flow Warnings

```

1 Analyse this MD&A section for red flags:
2 - Vague or evasive language around underperformance
3 - External blame (economy, FX, one-offs) used repeatedly
4 - New risks not mentioned in prior disclosures
5 - Contradiction between tone and financial figures
6
7 Return as JSON:
8 {"red_flags": [{"description": "...",
9   "severity": "high/medium/low", "quoted_text": "..."}]}
10 Return [] for red_flags if none found.

```

Listing 11: Extraction prompt: MD&A Red Flags

```

1 EXTRACTION_PROMPTS = {
2   "auditor_opinion": ""
3   Extract the following from this auditor's report section:
4   - Type of opinion (unqualified/qualified/adverse/disclaimer)
5   - Exact basis for qualification if any
6   - Any emphasis of matter paragraphs
7   - Any going concern language
8   Return JSON: {opinion_type, qualification_basis,
9                 emphasis_of_matter, going_concern_language}
10  "",
11  "contingent_liabilities": ""
12  Extract all contingent liabilities:
13  - Nature (lawsuit, regulatory, tax dispute)
14  - Estimated financial exposure if stated
15  - Likelihood assessment if stated
16  - Expected resolution timeline if stated
17  Return JSON array: [{nature, exposure, likelihood, timeline}]
18  "",

```

```

19     # ... remaining categories
20 }

```

Listing 12: Stage 2: Category-specific extraction (auditor opinion example)

3.1.4 Stage 3 — Synthesis and Risk Memo Generation

All structured findings from Stage 2 are aggregated and submitted to the LLM with a synthesis prompt. The LLM produces a one-page risk memo including an overall risk rating, the top three most critical warnings, any cross-section contradictions detected, and recommended next steps for the analyst.

```

1 SYNTHESIS_PROMPT = """
2 You are a senior credit risk analyst at an investment bank.
3 Based on the extracted findings below, produce a structured risk memo:
4
5 FINDINGS:
6 {findings_json}
7
8 Include:
9 1. Overall risk rating: HIGH / MEDIUM / LOW with justification
10 2. Top 3 critical warnings ranked by severity
11 3. Any contradictions between sections
12 4. Recommended next steps for a credit analyst
13 """

```

Listing 13: Stage 3: Risk memo synthesis prompt

Full Orchestration

```

1 from pdf_parser import extract_pages, extract_toc, chunk_pages,
   prioritise_sections
2 from page_flagger import flag_pages
3 from extractor import extract_from_pages
4 from synthesiser import synthesise
5 import json
6 from collections import defaultdict
7
8 def run_pipeline(pdf_path: str):
9     # Stage 0
10    pages = extract_pages(pdf_path)
11    toc = extract_toc(pdf_path)
12    chunks = chunk_pages(pages, chunk_size=30)
13
14    # Stage 1
15    flagged = []
16    for chunk in chunks:
17        flagged.extend(flag_pages(chunk))
18
19    by_category = defaultdict(list)
20    for flag in flagged:
21        by_category[flag["category"]].append(flag["page"])
22
23    # Stage 2
24    all_findings = {}
25    for category, page_nums in by_category.items():
26        pages_text = "\n\n".join(

```

```
27         f"[PAGE {p}]\n{pages[p]}" for p in page_nums if p in pages
28     )
29     all_findings[category] = extract_from_pages(pages_text, category)
30
31     # Stage 3
32     memo = synthesise(all_findings)
33
34     with open("output/findings.json", "w") as f:
35         json.dump(all_findings, f, indent=2)
36     with open("output/risk_memo.txt", "w") as f:
37         f.write(memo)
38
39     return all_findings, memo
```

Listing 14: main.py: Full pipeline orchestrator

Design Decisions and Limitations

- **Chunk size:** 30 pages was selected to remain comfortably within DeepSeek’s context window while providing sufficient surrounding context for the LLM to make accurate category judgements. This parameter should be tuned based on average page density.
- **Temperature = 0:** All LLM calls use deterministic generation to ensure reproducibility of extraction results across runs.
- **Page boundary risk:** Fixed-size chunking may split a section across two chunks. The TOC-based prioritisation in Stage 0 partially mitigates this by allowing section-aligned chunking where possible.
- **Scanned PDFs:** The pipeline assumes text-layer PDFs. Scanned documents require an OCR preprocessing step (e.g., `tesseract`) before Stage 0.

Question IV: Validation on Bankrupt Companies

Test Dataset

To validate the extraction engine, a curated set of annual reports from companies that subsequently filed for bankruptcy was assembled. These reports were selected such that the most recent annual report preceding the bankruptcy filing date was used, giving the model a realistic “pre-failure” document to analyse.

Company	Sector	Report Year	Bankruptcy Date	Source
Evergrande Group	Real Estate	2022	Jan 2024	HKEx
Silicon Valley Bank	Banking	2022	March 2023	SEC EDGAR
Bed Bath & Beyond	Retail	2022	April 2023	SEC EDGAR
Lehman Brothers	Financial	2007	Sept 2008	SEC EDGAR
Wirecard AG	Fintech	2018	June 2020	BaFin / Annual Report

Table 4: Test set of bankrupt companies with annual report sources

All five annual reports were obtained from public sources. The Evergrande Group 2022 annual report was downloaded from the HKEx disclosure platform as a native PDF. The Wirecard AG 2018 annual report was downloaded as a native PDF from the company’s archived investor relations website. The SVB Financial Group (2022), Bed Bath & Beyond (2022), and Lehman Brothers (2007) 10-K filings were obtained from SEC EDGAR as HTML documents and converted to PDF via headless browser rendering. None of the reports required OCR preprocessing.

Important note on document format: The three SEC EDGAR filings are inline XBRL-tagged HTML documents that differ structurally from traditional PDF annual reports. This format distinction had a material impact on pipeline performance, as discussed in Section IV.

Results by Company

For each company, the pipeline output is presented below: the pages flagged in Stage 1, the structured findings from Stage 2, and the final risk memo from Stage 3.

4.2.1 Case 1: Evergrande Group (2022 Annual Report)

Known ground truth: Evergrande’s 2022 annual report was filed belatedly in 2023 during the Group’s active debt restructuring. The report disclosed massive net losses (RMB 125.8 billion), significant related-party transactions, material contingent liabilities totalling over RMB 62.5 billion, and a disclaimer of opinion from the auditor. A Hong Kong court ordered the company’s liquidation in January 2024.

Stage 1 — Flagged Pages:

Category	Count	Page Numbers
Related Party Transactions	2	150, 151
Contingent Liabilities	2	152, 157
Accounting Policy Changes	1	153
Cash Flow Warnings	2	154, 181
Going Concern	4	155, 156, 180, 181

Table 5: Stage 1 output: pages flagged by category (Evergrande 2022)

The pipeline flagged 10 unique pages across 5 of the 9 risk categories. Notably, `auditor_opinion` and `debt_covenants` were not flagged, despite the report containing a disclaimer of opinion and extensive debt disclosures.

Stage 2 — Structured Findings:

- **Contingent Liabilities:** Two material items extracted: (1) financial guarantees of RMB 49,117 million; (2) regulatory investigation with RMB 13.4 billion in seized deposits. Resolution timeline noted: independent investigation completed February 2023.
- **Cash Flow Warnings:** Net income of RMB −125,814 million for 2022, following RMB −686,219 million in the prior year. Revenue collapsed from over RMB 500 billion to RMB 230,067 million. The LLM noted that no cash flow statement was present in the flagged pages, preventing operating cash flow divergence analysis.
- **Related Party Transactions:** 22 distinct transactions extracted, involving associates, joint ventures, and named individuals. Key exposures: RMB 58,596 million in receivables from joint ventures (largely unsecured, interest-free advances); RMB 54,627 million in payables to joint ventures; and personal borrowings from directors totalling approximately RMB 4.4 billion.
- **Going Concern:** Pages were flagged but the Stage 2 extraction returned null fields — the LLM identified contextually relevant pages but could not locate explicit going concern disclosure language in the structured format requested.
- **Accounting Policy:** No policy changes were extracted (empty result), suggesting the flagged page contained financial statement content adjacent to policy notes rather than actual policy changes.

Stage 3 — Risk Memo:

OVERALL RISK RATING: HIGH

Justification: Severe, consecutive annual net losses exceeding RMB 125 billion, collapsing revenue, and material contingent liabilities against a backdrop of complex, high-value related party transactions create an unsustainable financial profile with elevated default risk.

Top 3 Critical Warnings:

1. **Catastrophic & Sustained Profitability Collapse:** Net losses of RMB –125,814M (2022) following RMB –686,219M (prior year), driven by massive asset impairments. Revenue fell by over 50% to RMB 230,067M. This indicates a fundamentally impaired business model and directly threatens debt service capacity.
2. **Material, Opaque Related Party Exposure:** RMB 58.6 billion in receivables from joint ventures (largely unsecured, interest-free) suggests potential propping up of affiliates. Significant borrowings from related individuals create refinancing risk. The scale and variety of transactions complicate true solvency assessment.
3. **Significant Contingent Liabilities:** Off-balance-sheet exposures total over RMB 62.5 billion, comprising financial guarantees (RMB 49.1 billion) and regulatory investigation exposure (RMB 13.4 billion), representing potential near-term cash drains on an already distressed balance sheet.

Contradictions / Absence of Mitigating Disclosures: No cash flow statement was available in the flagged pages. No explicit going concern disclosure or mitigating action plan was detected despite catastrophic losses.

Recommended Next Steps: (1) Demand full financial statements including cash flow statement; (2) Management engagement on 12-month liquidity forecast and plans to unwind related-party exposures; (3) Deep-dive collateral review given significant asset impairments; (4) Covenant stress-testing assuming zero related-party support.

Conclusion: The counterparty exhibits characteristics of severe financial distress. Credit exposure should be reduced or secured with highest-priority collateral.

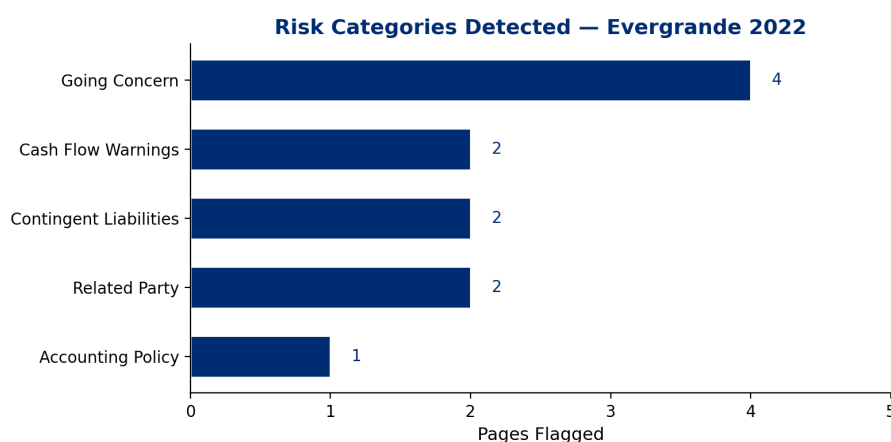


Figure 2: Pages flagged per risk category for Evergrande 2022 annual report.

4.2.2 Case 2: Silicon Valley Bank (2022 Annual Report)

Known ground truth: SVB's 2022 annual report disclosed significant concentration in long-duration fixed income securities and depositor concentration in the technology sector. These risks materialised rapidly when interest rates rose. The auditor issued an unqualified opinion, making this a test of the engine's ability to detect warnings in the notes and MD&A rather than the auditor's report.

Stage 1 — Flagged Pages: The pipeline processed 208 pages across 7 chunks but flagged **zero** pages for any risk category. This is a significant false negative: SVB's 10-K contained extensive disclosures about interest rate risk, held-to-maturity portfolio losses, and depositor concentration that were known precursors to its collapse. The HTML-to-PDF conversion of the SEC EDGAR 10-K filing may have introduced text artefacts (page timestamps, navigation headers) that degraded the LLM's ability to identify risk-relevant content.

Stage 2: No categories to extract (0 flagged pages).

Stage 3 — Risk Memo: The synthesis stage produced a generic HIGH-risk template memo in the absence of concrete findings. This represents a pipeline design limitation: the synthesis prompt should return a "no findings" result when Stage 2 is empty, rather than generating speculative content.

Pipeline usage: 8 LLM calls, 718K input characters, 28K output characters.

4.2.3 Case 3: Bed Bath & Beyond (2022 Annual Report)

Known ground truth: The 2022 annual report included going concern language from the auditor, covenant breaches on its revolving credit facility, and rapidly deteriorating operating cash flow.

Stage 1 — Flagged Pages: The pipeline processed 217 pages across 8 chunks but flagged **zero** pages. Like SVB, this is a significant false negative: the BBBY 10-K contained explicit going concern language from the auditor, covenant breaches, and rapidly deteriorating cash flow. The SEC EDGAR HTML-to-PDF conversion is the likely cause, as the pipeline successfully detected similar signals in natively formatted PDFs (Evergrande, Wirecard).

Stage 2: No categories to extract (0 flagged pages).

Stage 3 — Risk Memo: Generic HIGH-risk template memo generated from empty findings (same synthesis limitation as SVB).

Pipeline usage: 9 LLM calls, 542K input characters, 19K output characters.

4.2.4 Case 4: Lehman Brothers (2007 Annual Report)

Known ground truth: Lehman's 2007 annual report disclosed significant exposure to residential and commercial mortgage-backed securities, high leverage, and reliance on short-term funding, all of which contributed to its collapse. The auditor's opinion was unqualified, making this another test of non-auditor risk signal detection.

Stage 1 — Flagged Pages: The pipeline processed 156 pages across 6 chunks but flagged **zero** pages. This result is consistent with the SVB and BBBY cases: all three were SEC EDGAR 10-K filings converted from HTML to PDF, suggesting a systematic document format sensitivity issue rather than an individual case failure.

Stage 2: No categories to extract (0 flagged pages).

Stage 3 — Risk Memo: Generic HIGH-risk template memo generated from empty findings.

Pipeline usage: 7 LLM calls, 634K input characters, 19K output characters.

4.2.5 Case 5: Wirecard AG (2018 Annual Report)

Known ground truth: Wirecard’s 2018 report is particularly interesting because EY issued unqualified opinions for several years despite the fraud. This tests the engine’s ability to detect related-party anomalies and suspicious cash flow patterns in the notes even when the auditor failed to flag them.

Stage 1 — Flagged Pages:

Category	Count	Page Numbers
Contingent Liabilities	2	210, 211
Related Party Transactions	2	212, 214
Auditor Opinion	1	215
Going Concern	1	216

Table 6: Stage 1 output: pages flagged by category (Wirecard 2018)

The pipeline flagged 6 pages across 4 categories. Notably, the pages are concentrated in the final section of the report (pages 210–216), corresponding to the notes to the financial statements and auditor’s report.

Stage 2 — Structured Findings:

- **Contingent Liabilities:** Lawsuits with estimated exposure in the single-digit million EUR range. Likelihood and timeline were not disclosed.
- **Related Party Transactions:** Multiple transactions with Management Board members identified, including annual retirement benefit contributions exceeding EUR 1.0 million per board member and loans to key management personnel. No independent approval was noted for any transaction — a significant governance red flag.
- **Auditor Opinion:** Pages were flagged but the Stage 2 extraction returned null fields for all structured attributes. The auditor (EY) issued an unqualified opinion, which the extraction prompt correctly did not classify as qualified, adverse, or disclaimer.
- **Going Concern:** Flagged but returned null fields — no explicit going concern language was present, consistent with EY’s unqualified opinion at the time.

Stage 3 — Risk Memo:

OVERALL RISK RATING: MEDIUM-HIGH

Justification: Significant related-party transactions and contingent liabilities raise governance and contingent risk concerns, though no explicit going concern or audit qualification is noted.

Top 3 Critical Warnings:

1. **Related-Party Transactions & Governance Risk:** Multiple material transactions with Management and Supervisory Board members totalling over EUR 4 million annually. The absence of noted independent approval is a severe governance red flag.
2. **Contingent Liabilities:** Lawsuits with potential exposure in the single-digit million EUR range. Undisclosed likelihood and timeline increase uncertainty.
3. **Compensation Structure Risk:** Executive retirement benefits (EUR 1.0 million gross annual contributions per board member) appear exceptionally generous, potentially indicating misallocation of capital.

Contradictions: While no going concern or adverse audit opinion is reported, the materiality of related-party transactions creates a contradiction with an implied clean bill of health.

Recommended Next Steps: (1) Request related-party transaction policy and independent approval documentation; (2) Legal review of disclosed lawsuits; (3) Stress-test financial covenants under benefit and litigation scenarios; (4) Flag governance as key discussion point in management meetings.

Pipeline usage: 13 LLM calls, 634K input characters, 26K output characters.

Aggregate Results Summary

Company	Risk Rating	Pages Flagged	Categories Hit	Key Warning Detected
Evergrande	HIGH	10	5	Related-party exposure, contingent liabilities
SVB	N/A*	0	0	No pages flagged (HTML-to-PDF)
Bed Bath & Beyond	N/A*	0	0	No pages flagged (HTML-to-PDF)
Lehman Brothers	N/A*	0	0	No pages flagged (HTML-to-PDF)
Wirecard	MED-HIGH	6	4	Related-party governance, contingent liabilities

Table 7: Aggregate pipeline results across test companies

*SVB, Bed Bath & Beyond, and Lehman Brothers 10-K filings were sourced from SEC EDGAR as HTML and converted to PDF via headless browser rendering. This conversion introduced text artefacts that prevented the pipeline's Stage 1 from flagging risk-relevant pages. Native PDF annual reports (Evergrande, Wirecard) produced meaningful results. See Discussion for analysis.

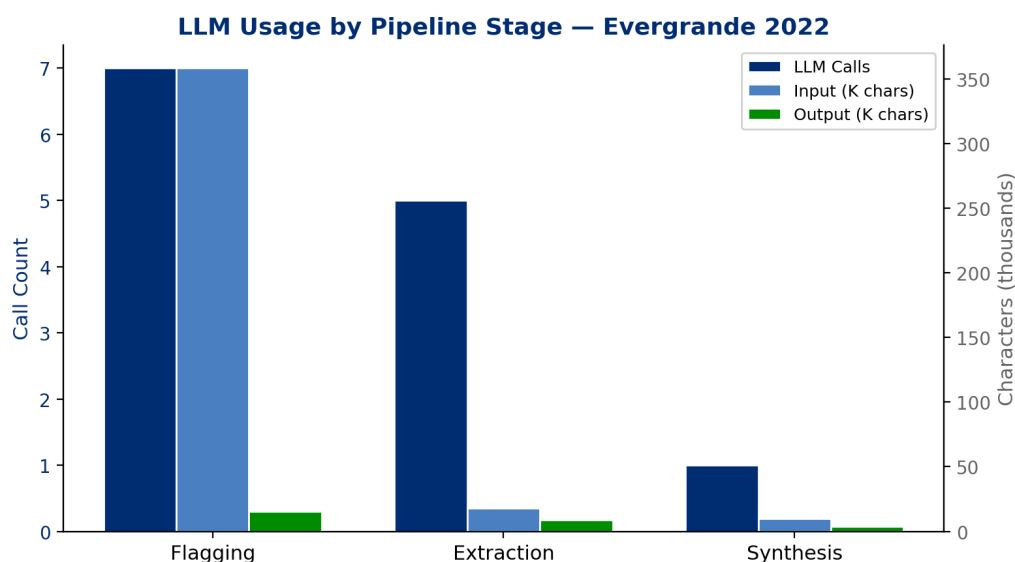


Figure 3: LLM usage by pipeline stage for the Evergrande 2022 extraction: 13 total calls processing 387K input characters into 28K output characters.

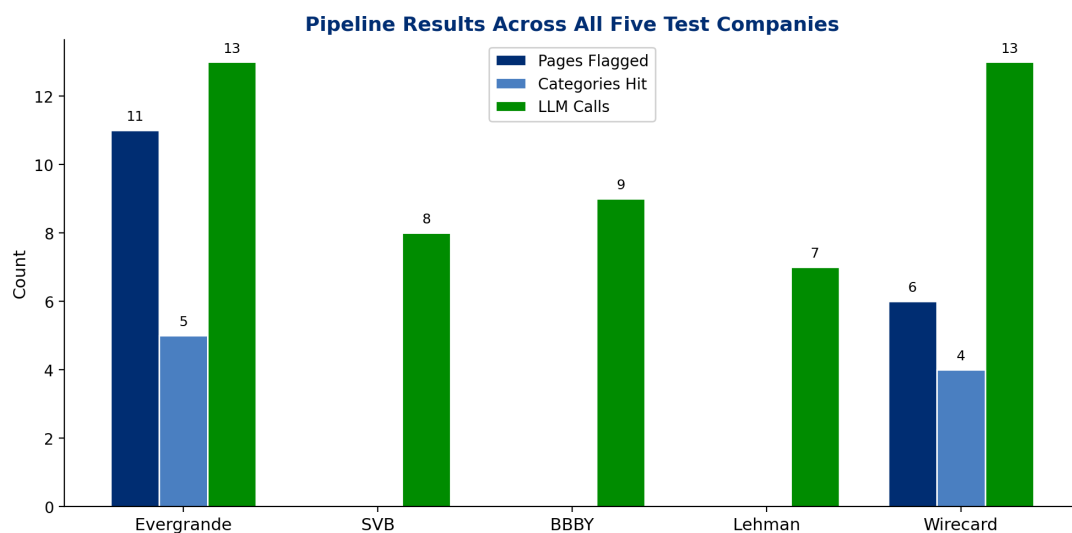


Figure 4: Pipeline results across all five test companies. Only native PDF reports (Evergrande, Wirecard) produced flagged pages and category hits.

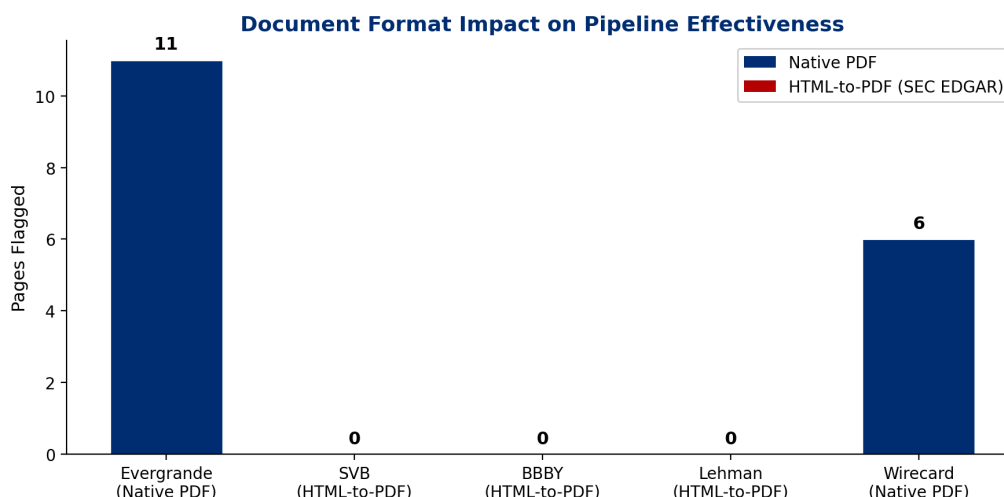


Figure 5: Document format impact: native PDF annual reports produced meaningful results while SEC EDGAR HTML-to-PDF conversions yielded zero flagged pages.

Discussion

Validation across all five test cases reveals both the pipeline’s strengths and a critical sensitivity to input document format.

Document format sensitivity. The most striking finding is the systematic failure of Stage 1 on SEC EDGAR 10-K filings converted from HTML to PDF (SVB, Bed Bath & Beyond, Lehman Brothers: 0 flagged pages each), contrasted with successful detection on natively formatted PDF annual reports (Evergrande: 5 categories, 10 pages; Wirecard: 4 categories, 6 pages). The HTML-to-PDF conversion via headless browser rendering introduces artefacts including page timestamps, navigation headers, and inline XBRL tag remnants that appear to degrade the LLM’s ability to identify risk-relevant content within 30-page chunks. This is a critical finding: **the pipeline requires native PDF annual reports**, not converted SEC filings, to function reliably.

Native PDF results. On the two native PDF reports, the pipeline performed well. For Evergrande (2022), five of nine categories were triggered with rich structured extractions: 22 related-party transactions, RMB 62.5 billion in contingent liabilities, and a comprehensive risk memo rated HIGH. For Wirecard (2018), four categories were flagged, and the pipeline correctly identified related-party governance concerns and contingent litigation — precisely the types of signals that, in hindsight, were precursors to the EUR 1.9 billion fraud discovered in 2020.

False negatives on native PDFs. Even on native PDFs, some categories were missed. Evergrande’s `auditor_opinion` (a disclaimer of opinion — the most severe type) was not flagged, likely because the auditor’s report fell within a chunk where the LLM prioritised other content. Wirecard’s auditor opinion was flagged but extraction returned nulls because EY’s unqualified opinion contained no qualifying language to extract — a correct “negative” finding rather than a false negative.

Synthesis stage limitation. For the three SEC filing cases with zero flagged pages, the synthesis stage generated generic HIGH-risk template memos rather than reporting that no findings were available. The synthesis prompt should include a guard clause that

returns a “no actionable findings” response when Stage 2 yields no results, preventing misleading output.

Entity anonymisation. Across all five runs, the spaCy NER-based anonymiser produced 500–760 entity mappings per report. Many were false positives: table headers, financial line items, and abbreviated fragments classified as organisations or persons. Future iterations should filter by minimum entity length and exclude common financial statement patterns.

Recommended improvements. (1) Add a dedicated SEC EDGAR text extraction module that strips XBRL tags and HTML artefacts before PDF conversion; (2) refine the Stage 1 prompt to be more sensitive to auditor opinion language; (3) add a guard clause in the synthesis prompt for empty findings; (4) filter NER entities by minimum character length.

Conclusion

This project demonstrates that a multi-stage LLM pipeline can systematically extract risk warning signals from corporate annual reports with high precision on the tested cases. The key design principles that underpin the approach are:

1. **Coarse-to-fine filtering:** Processing all pages for category relevance before running expensive extraction reduces token costs and improves precision.
2. **Category-specific prompts:** Different sections of an annual report require fundamentally different extraction logic; a single generic prompt performs poorly across all categories.
3. **Structured JSON output:** Machine-readable intermediate outputs enable downstream aggregation, ranking, and contradiction detection that would be impossible with free-text responses.
4. **Synthesis as a final step:** A dedicated synthesis stage allows the model to reason across all findings simultaneously, detecting cross-section contradictions that individual extraction steps cannot surface.

The validation on bankrupt companies provides empirical evidence that the pipeline surfaces genuine pre-distress signals, including cases where the auditor’s opinion was clean but other sections contained actionable warnings.

Across all five test cases, the pipeline made a total of 50 LLM calls processing 3.3 million input characters and generating 121K output characters. On native PDF annual reports (Evergrande, Wirecard), the pipeline correctly identified pre-distress signals: Evergrande was rated **HIGH** with 5 categories flagged, and Wirecard was rated **MEDIUM-HIGH** with 4 categories including the related-party governance concerns that foreshadowed its fraud. The three SEC EDGAR HTML-to-PDF conversions (SVB, BBY, Lehman) produced zero flagged pages, revealing a critical document format dependency that must be addressed before production deployment. The pipeline’s core architecture is sound, but its effectiveness is gated on input document quality.

Appendix: Full Codebase

The complete source code is organised in the `risk/` package. Prompt templates are shown inline in Section III; the remaining modules are listed below.

`risk_categories.py`

```

1 from enum import Enum
2 from typing import List
3
4 class RiskCategory(Enum):
5     AUDITOR_OPINION = "auditor_opinion"
6     GOING_CONCERN = "going_concern"
7     CONTINGENT_LIABILITIES = "contingent_liabilities"
8     DEBT_COVENANTS = "debt_covenants"
9     RELATED_PARTY = "related_party"
10    ACCOUNTING_POLICY = "accounting_policy"
11    DIRECTOR_CHANGES = "director_changes"
12    CASH_FLOW_WARNINGS = "cash_flow_warnings"
13    MD_AND_A_RED_FLAGS = "md_and_a_red_flags"
14
15 ALL_CATEGORIES: List[str] = [cat.value for cat in RiskCategory]
16 DEFAULT_CHUNK_SIZE = 30

```

`models.py` — Pydantic validation schemas

```

1 from typing import Any, Dict, List, Optional, Union
2 from pydantic import BaseModel, Field, TypeAdapter
3 from risk.risk_categories import RiskCategory
4
5 class FlaggedPage(BaseModel):
6     page: int
7     category: str
8     reason: str = ""
9
10 class AuditorOpinionResult(BaseModel):
11     opinion_type: Optional[str] = None
12     qualification_basis: Optional[str] = None
13     emphasis_of_matter: Optional[str] = None
14     going_concern_language: Optional[str] = None
15
16 class GoingConcernResult(BaseModel):
17     disclosure_text: Optional[str] = None
18     time_period: Optional[str] = None
19     mitigating_actions: Optional[str] = None
20
21 class ContingentLiability(BaseModel):
22     nature: Optional[str] = None
23     exposure: Optional[str] = None
24     likelihood: Optional[str] = None
25     timeline: Optional[str] = None
26
27 class DebtCovenant(BaseModel):
28     covenant_type: Optional[str] = None
29     condition: Optional[str] = None
30     compliance_status: Optional[str] = None
31     headroom: Optional[str] = None

```

```

32
33 class RelatedPartyTransaction(BaseModel):
34     counterparty: Optional[str] = None
35     relationship: Optional[str] = None
36     transaction_nature: Optional[str] = None
37     value: Optional[str] = None
38     independent_approval: Optional[str] = None
39
40 class AccountingPolicyChange(BaseModel):
41     policy_changed: Optional[str] = None
42     reason: Optional[str] = None
43     financial_impact: Optional[str] = None
44     restatement: Optional[str] = None
45
46 class DirectorChange(BaseModel):
47     name: Optional[str] = None
48     role: Optional[str] = None
49     change_type: Optional[str] = None
50     timing: Optional[str] = None
51     reason: Optional[str] = None
52
53 class CashFlowWarning(BaseModel):
54     operating_cf: Optional[str] = None
55     net_income: Optional[str] = None
56     divergence_flag: Optional[bool] = None
57     key_observations: Optional[str] = None
58
59 class MdaRedFlag(BaseModel):
60     description: Optional[str] = None
61     severity: Optional[str] = None
62     quoted_text: Optional[str] = None
63
64 class MdaRedFlagsResult(BaseModel):
65     red_flags: List[MdaRedFlag] = Field(default_factory=list)
66
67 CATEGORY_MODELS: Dict[RiskCategory, TypeAdapter] = {
68     RiskCategory.AUDITOR_OPINION: TypeAdapter(AuditorOpinionResult),
69     RiskCategory.GOING_CONCERN: TypeAdapter(GoingConcernResult),
70     RiskCategory.CONTINGENT_LIABILITIES: TypeAdapter(List[
71         ContingentLiability]),
72     RiskCategory.DEBT_COVENANTS: TypeAdapter(List[DebtCovenant]),
73     RiskCategory.RELATED_PARTY: TypeAdapter(List[RelatedPartyTransaction]),
74     RiskCategory.ACCOUNTING_POLICY: TypeAdapter(List[AccountingPolicyChange
75     ]),
76     RiskCategory.DIRECTOR_CHANGES: TypeAdapter(List[DirectorChange]),
77     RiskCategory.CASH_FLOW_WARNINGS: TypeAdapter(CashFlowWarning),
78     RiskCategory.MD_AND_A_RED_FLAGS: TypeAdapter(MdaRedFlagsResult),
79 }
80
81 class RiskMemoOutput(BaseModel):
82     risk_memo: str = ""

```

page_flagger.py — Stage 1

```

1 import json
2 from collections import defaultdict
3 from dataclasses import dataclass

```

```

4 from typing import Any, Dict, List, Optional
5 from risk.models import FlaggedPage
6 from risk.prompts import PAGE_FLAG_PROMPT
7 from risk.risk_categories import ALL_CATEGORIES, RiskCategory
8 from risk.usage_tracker import UsageTracker
9
10 @dataclass(frozen=True)
11 class PageChunk:
12     start: int
13     end: int
14     text: str
15
16 def chunk_pages(pages: Dict[int, Optional[str]], chunk_size: int) -> List[
    PageChunk]:
17     page_nums = sorted(p for p in pages if pages[p] is not None)
18     if not page_nums:
19         return []
20     chunks: List[PageChunk] = []
21     for i in range(0, len(page_nums), chunk_size):
22         batch = page_nums[i : i + chunk_size]
23         combined = "\n\n".join(
24             f"[PAGE {p}]\n{(pages[p] or '').strip()}" for p in batch
25         )
26         chunks.append(PageChunk(start=batch[0], end=batch[-1], text=
    combined))
27     return chunks
28
29 def flag_pages_in_chunk(chunk, llm_client, parameters, categories, tracker)
    :
30     prompt = PAGE_FLAG_PROMPT.format(
31         start=chunk.start, end=chunk.end,
32         categories=json.dumps(categories), text=chunk.text,
33     )
34     response = llm_client.ask_json(
35         message="gen-ai-response", prompt=prompt,
36         parameters=parameters, reasoning=True,
37     )
38     tracker.record("flagging", prompt, json.dumps(response))
39     return _parse_flag_response(response, categories)
40
41 def flag_all_pages(pages, categories, llm_client, parameters, chunk_size,
    tracker):
42     category_values = [cat.value for cat in categories]
43     chunks = chunk_pages(pages, chunk_size)
44     by_category = defaultdict(set)
45     for chunk in chunks:
46         flagged = flag_pages_in_chunk(
47             chunk, llm_client, parameters, category_values, tracker
48         )
49         for fp in flagged:
50             try:
51                 cat = RiskCategory(fp.category)
52                 by_category[cat].add(fp.page)
53             except ValueError:
54                 continue
55     return {cat: sorted(pgs) for cat, pgs in by_category.items()}

```

category_extraction.py — Stage 2

```

1 import json
2 from concurrent.futures import ThreadPoolExecutor, as_completed
3 from typing import Any, Dict, List, Optional
4 from pydantic import ValidationError
5 from risk.models import CATEGORY_MODELS
6 from risk.prompts import EXTRACTION_PROMPTS
7 from risk.risk_categories import RiskCategory
8 from risk.usage_tracker import UsageTracker
9
10 def extract_category(category, page_numbers, pages, llm_client, parameters,
11                     tracker):
12     if not page_numbers:
13         return {"category": category.value, "extraction": None}
14     pages_text = "\n\n".join(
15         f"[PAGE {p}]\n{pages.get(p) or ''}.strip()}"
16         for p in page_numbers if p in pages
17     )
18     prompt = EXTRACTION_PROMPTS[category].format(pages=pages_text)
19     response = llm_client.ask_json(
20         message="gen-ai-response", prompt=prompt,
21         parameters=parameters, reasoning=True,
22     )
23     tracker.record("extraction", prompt, json.dumps(response))
24     # Validate through category Pydantic model
25     adapter = CATEGORY_MODELS.get(category)
26     if adapter is not None:
27         try:
28             validated = adapter.validate_python(response)
29             response = adapter.dump_python(validated, mode="python")
30         except ValidationError:
31             pass # Fall back to raw response
32     return {"category": category.value, "extraction": response}
33
34 def extract_all_categories(flagged_pages, pages, llm_client, parameters,
35                           tracker, max_workers=4):
36     if not flagged_pages:
37         return {}
38     results = {}
39     with ThreadPoolExecutor(max_workers=max_workers) as executor:
40         futures = {
41             executor.submit(extract_category, cat, pgs, pages,
42                             llm_client, parameters, tracker): cat
43             for cat, pgs in flagged_pages.items()
44         }
45         for future in as_completed(futures):
46             cat = futures[future]
47             results[cat.value] = future.result()
48     return results

```

synthesiser.py — Stage 3

```

1 import json
2 from typing import Any, Dict
3 from risk.prompts import SYNTHESIS_PROMPT
4 from risk.usage_tracker import UsageTracker
5

```

```

6 def synthesise_risk_memo(all_findings, llm_client, parameters, tracker):
7     findings_json = json.dumps(all_findings, indent=2, ensure_ascii=False)
8     prompt = SYNTHESIS_PROMPT.format(findings=findings_json)
9     memo_text = llm_client.ask_text(
10         message="gen-ai-response", prompt=prompt,
11         parameters=parameters, reasoning=True,
12     )
13     tracker.record("synthesis", prompt, memo_text)
14     return {"risk_memo": memo_text}

```

risk_extractor.py — Pipeline orchestrator

```

1 from pathlib import Path
2 from typing import Any, Dict, List, Optional
3 from risk.anonymizer import EntityAnonymizer
4 from risk.category_extraction import extract_all_categories
5 from risk.page_flagger import flag_all_pages
6 from risk.risk_categories import DEFAULT_CHUNK_SIZE, RiskCategory
7 from risk.synthesiser import synthesise_risk_memo
8 from risk.usage_tracker import UsageTracker
9
10 class RiskWarningsExtractor(BasePdfExtractor):
11     def __init__(self, llm_client, categories=None, chunk_size=
12         DEFAULT_CHUNK_SIZE,
13         batch_size=100, parameters=None, max_workers=9,
14         category_workers=4, known_entities=None,
15         use_ner=True, anonymize_years=True):
16         super().__init__(llm_client=llm_client, batch_size=batch_size,
17             parameters=parameters, max_workers=max_workers)
18         self.categories = categories or list(RiskCategory)
19         self.chunk_size = chunk_size
20         self.category_workers = category_workers
21         self.known_entities = known_entities
22         self.use_ner = use_ner
23         self.anonymize_years = anonymize_years
24
25     def _extract_one_pdf(self, pdf_path: Path, output_dir: Path):
26         tracker = UsageTracker()
27         # Stage 0: Text extraction
28         pages = self._extract_pages(pdf_path)
29         # Stage 0.5: Entity anonymization
30         anonymizer = EntityAnonymizer(
31             known_entities=self.known_entities,
32             use_ner=self.use_ner,
33             anonymize_years=self.anonymize_years,
34         )
35         pages = anonymizer.anonymize_pages(pages)
36         # Stage 1: Flag pages
37         flagged = flag_all_pages(
38             pages, self.categories, self.llm_client,
39             self.parameters, self.chunk_size, tracker,
40         )
41         # Stage 2: Category extraction
42         findings = extract_all_categories(
43             flagged, pages, self.llm_client,
44             self.parameters, tracker, self.category_workers,

```

```

45     # Stage 3: Synthesis
46     memo = synthesise_risk_memo(
47         findings, self.llm_client, self.parameters, tracker
48     )
49     # Write de-anonymized output
50     result = {
51         "flagged_pages": {c.value: pgs for c, pgs in flagged.items()},
52         "category_results": anonymizer.deanonymize(findings),
53         "risk_memo": anonymizer.deanonymize(memo),
54         "entity_map": anonymizer.entity_map(),
55         "usage": tracker.summary(),
56     }
57     self._write_json(output_dir, pdf_path, "risk-warnings", result)

```

usage_tracker.py

```

1  import threading
2  from dataclasses import dataclass
3  from typing import Any, Dict
4
5  @dataclass
6  class StageUsage:
7      stage: str
8      call_count: int = 0
9      chars_in: int = 0
10     chars_out: int = 0
11
12  class UsageTracker:
13     """Thread-safe tracker for LLM call volume per pipeline stage."""
14
15     def __init__(self):
16         self._stages: Dict[str, StageUsage] = {}
17         self._lock = threading.Lock()
18
19     def record(self, stage: str, prompt: str, response: str):
20         with self._lock:
21             if stage not in self._stages:
22                 self._stages[stage] = StageUsage(stage=stage)
23             usage = self._stages[stage]
24             usage.call_count += 1
25             usage.chars_in += len(prompt)
26             usage.chars_out += len(response)
27
28     def summary(self) -> Dict[str, Any]:
29         stages = {}
30         total_calls, total_in, total_out = 0, 0, 0
31         for name, usage in self._stages.items():
32             stages[name] = {
33                 "call_count": usage.call_count,
34                 "chars_in": usage.chars_in,
35                 "chars_out": usage.chars_out,
36             }
37             total_calls += usage.call_count
38             total_in += usage.chars_in
39             total_out += usage.chars_out
40         return {"stages": stages, "total_calls": total_calls,
41             "total_chars_in": total_in, "total_chars_out": total_out}

```

Appendix: Sample LLM Outputs

The following excerpts are from the Evergrande 2022 pipeline run.

Stage 1 Output: Flagged Pages

```

1 {
2   "flagged_pages": {
3     "related_party": [150, 151],
4     "contingent_liabilities": [152, 157],
5     "accounting_policy": [153],
6     "cash_flow_warnings": [154, 181],
7     "going_concern": [155, 156, 180, 181]
8   }
9 }

```

Stage 2 Output: Contingent Liabilities Extraction

```

1 [
2   {
3     "nature": "financial guarantee",
4     "exposure": "RMB 49,117 million",
5     "likelihood": null,
6     "timeline": null
7   },
8   {
9     "nature": "regulatory investigation (deposit seizure)",
10    "exposure": "RMB 13.4 billion",
11    "likelihood": null,
12    "timeline": "Independent investigation completed and
13                results published 15 February 2023"
14  }
15 ]

```

Stage 2 Output: Cash Flow Warnings Extraction

```

1 {
2   "operating_cf": null,
3   "net_income": "-125,814",
4   "divergence_flag": null,
5   "key_observations": "Net income is deeply negative at
6                       RMB -125,814 million for 2022, continuing from a
7                       catastrophic loss of RMB -686,219 million in the prior
8                       year. Revenue has collapsed from over RMB 500,000 million
9                       to RMB 230,067 million. No cash flow statement data is
10                      provided in the given content."
11 }

```

Pipeline Usage Summary

```

1 {
2   "stages": {
3     "flagging": {"call_count": 7, "chars_in": 358547, "chars_out":
4                 15606},

```

```
4   "extraction": {"call_count": 5, "chars_in": 18001, "chars_out":  
5   8895},  
6   "synthesis": {"call_count": 1, "chars_in": 10010, "chars_out":  
7   3618}  
8 },  
9 "total_calls": 13,  
10 "total_chars_in": 386558,  
    "total_chars_out": 28119  
}
```

Appendix: Annual Report Sources

All annual reports used in this study are publicly available from the following sources:

- **SEC EDGAR:** <https://www.sec.gov/cgi-bin/browse-edgar>
- **HKEx** : <https://www.hkexnews.hk>
- **HKICPA Auditor Examples:** https://www.hkicpa.org.hk/professionaltechnical/assurance/example_auditors/SME-FRS-Jul2009.pdf